University of Rwanda

Information Systems

Group 1

GASORE Imena Kenny	**225024943**
BAKUNZI Egide	**225053944**
NIYOGISUBIZO Aline	**225037986**
KUNDA UWAYO Kellia	**225039228**
BERABOSE UWACU Onesme	**225034054**
UWINEZA Natacha	**225075470**
UMUHIRE MARIE Brillance	**225072492**
MFITIMANA Eric	**225069400**

OS Assignment

Implementation of Interprocess Communication and Synchronization in Java

Mainly regarding cooperating processes or threads, as they share resources and acquire synchronization mechanisms like semaphore, mutexes and monitors to prevent errors and ensure proper ordering:

1. **Race condition & Mutual Exclusion (Mutex)**

   When multiple threads access shared memory concurrently without synchronization, a race condition occurs because read-modify-write operations are not atomic. We can fix this using a Mutex (Mutual Exclusion lock).

   The Following are just snippets of essential parts of the java program that runs two threads simultaneously to increment a shared memory to 100000 :

   Imports:

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
```

   Unsafe incrementation

```
public void increment1() {
    int temp = counter;
    Thread.yield();
    counter = temp + 1;
}
```

   Safe incrementation:

```
public void increment2() {
    lock.lock();
    try {
        counter++;
    } finally {
        lock.unlock();
    }
}
```

   Without locks, simultaneous thread reads and writes overwrite each other (lost updates), yielding corrupted total.

```
Thread thread1 = new Thread(() -> {
    for (int i = 0; i < ite; i++) {
        raceCondition.increment1();
    }
});

Thread thread2 = new Thread(() -> {
    for (int i = 0; i < ite; i++) {
        raceCondition.increment1();
    }
});

thread1.start();
thread2.start();

thread1.join();
thread2.join();
```

   This program runs two threads simultaneously which want to increment a shared memory (counter) to 100000 but without protection, there can be interferences leading to false outcomes.

   Unsafe output:

```
Actual Counter (Unsafe with Race Condition): 103150
```

   Safe output:

```
Final Counter Value using Mutex: 200000
```

2. **Producer-Consumer Problem using Semaphores**

   The bounded buffer problem requires producers to wait if the buffer is full and consumers to wait if it is empty, coordinating access via three semaphores (mutex, empty, and full)

Semaphore setup:

```
private static final Semaphore empty = new Semaphore(BUFFER_SIZE);
private static final Semaphore full = new Semaphore(permits: 0);
private static final Semaphore mutex = new Semaphore(permits: 1);
```

- **empty:** A counting semaphore initialized to the total number of empty slots in the buffer. It stops the producer if the buffer is full.
- **full:** A counting semaphore initialized to 0. It stops the consumer if the buffer is empty.
- **mutex:** A binary semaphore (initialized to 1) used to acquire and release the lock so only one thread can modify the buffer at a time.

The producer Thread:

```
static class Producer implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < NUM_ITEMS; i++) {
            try {
                empty.acquire();
                mutex.acquire();
                buffer[in] = i;
                System.out.println("Produced: " + i);
                in = (in + 1) % BUFFER_SIZE;
                mutex.release();
                full.release();
                Thread.sleep(millis: 300);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```

empty.acquire(): Decrements the empty count. If the buffer is full (empty == 0), the producer blocks and waits.

mutex.acquire(): Locks the critical section to ensure safe access.

mutex.release() & **full.release():** Unlocks the mutex and increments the full semaphore so the consumer knows an item is ready.

The consumer Thread:

```
static class Consumer implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < NUM_ITEMS; i++) {
            try {
                full.acquire();
                mutex.acquire();

                int item = buffer[out];
                System.out.println("Consumed: " + item);
                out = (out + 1) % BUFFER_SIZE;

                mutex.release();
                empty.release();
                Thread.sleep(millis: 600);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```

full.acquire(): Decrements the full count. If the buffer is empty (full == 0), the consumer blocks and waits.

mutex.acquire(): Locks the critical section.

mutex.release() & **empty.release():** Unlocks the mutex and increments the empty semaphore, freeing up space for the producer

3. **Monitor Implementation**

A Monitor is a high-level synchronization construct that encapsulates shared variables, mutual exclusion, and condition variables (wait and signal). In Java, this is natively supported using **synchronized, wait(), and notifyAll()**

```
static class BufferMonitor {
    private static final int BUFFER_SIZE = 5;
    private static int[] buffer = new int[BUFFER_SIZE];
    private static int in = 0;
    private static int out = 0;
    private static int count = 0;
```

BufferMonitor: This class acts as our Monitor. All shared data **(buffer, in, out, count)** is hidden safely inside it.

count: Keeps track of how many items are currently in the buffer so we know when it's full or empty.

```
public synchronized void produce(int item) throws InterruptedException {
    while (count == BUFFER_SIZE) {
        wait();
    }
    buffer[in] = item;
    System.out.println("Monitor Produced: " + item + " (Count: " + count + ")");
    in = (in + 1) % BUFFER_SIZE;
    count++;
    notifyAll();
}
```

```
public synchronized int consume() throws InterruptedException {
    while (count == 0) {
        wait();
    }
    int item = buffer[out];
    System.out.println("Monitor Consumed: " + item + " (Count: " + count + ")");
    out = (out + 1) % BUFFER_SIZE;
    count--;
    notifyAll();
    return item;
}
```

synchronized: ensures that **only one thread** can execute produce() or consume() at a time. Mutual exclusion is handled automatically by Java.

If the buffer is full, the producer calls **wait()**. This suspends the producer and automatically releases the monitor lock so other threads aren't blocked forever.

If the buffer is empty, the consumer calls **wait()** and goes to sleep until an item is produced.

notifyAll(): Wakes up any sleeping threads (like a waiting consumer) to let them know the state of the buffer has changed.

4. **Dining Philosophers Algorithm**

The Dining Philosophers problem illustrates the challenges of allocating limited resources (chopsticks) among multiple processes without causing deadlock or starvation.

Deadlock prevention snippet:

```
public Philosopher(int id) {
    this.id = id;

    int leftIndex = id;
    int rightIndex = (id + 1) % NUM_PHILOSOPHERS;

    if (leftIndex < rightIndex) {
        this.firstChopstick = chopSticks[leftIndex];
        this.secondChopstick = chopSticks[rightIndex];
    } else {
        this.firstChopstick = chopSticks[rightIndex];
        this.secondChopstick = chopSticks[leftIndex];
    }
}
```

It identifies the left chopstick index (**id**) and right chopstick index **((id + 1) % 5).**

Resource Hierarchy Logic: It checks which index is smaller (**leftIndex < rightIndex**). Whichever chopstick has the lower number is assigned to **firstChopstick,** and the higher-numbered one becomes **secondChopstick.**

By forcing everyone to grab the lower-numbered chopstick first, it breaks the circular waiting chain and prevents deadlock.

```
@Override
public void run() {
    try {
        for (int round = 1; round <= 2; round++) {
            System.out.println("Philosopher " + id + " is thinking.");
            Thread.sleep(millis: 1000);

            System.out.println("Philosopher " + id + " is hungry.");
            firstChopstick.acquire();
            secondChopstick.acquire();

            System.out.println("Philosopher " + id + " is eating (Round " + round + ").");
            Thread.sleep(millis: 1000);

            secondChopstick.release();
            firstChopstick.release();

            System.out.println("Philosopher " + id + " finished eating and put down chopsti
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
```

What it does: - Simulates thinking using **Thread.sleep(1000),** Calls **.acquire()** on **firstChopstick** and then **secondChopstick**. If a chopstick is already in use by a neighbor, the philosopher safely waits. Calls **.release()** on both chopsticks to hand them back so waiting neighbors can use them.